

Error Handling Best Practices

Overview

This document outlines best practices for implementing robust error handling in workflow automation. Following these guidelines will help ensure workflows are resilient, debuggable, and maintainable.

1. Workflow Execution Management

Execute Pin

Purpose: Preserve node output data without re-execution

When to use:

- When you need to reference previous node outputs
- To avoid unnecessary API calls or computations
- During debugging to maintain consistent data states

How to implement:

- Pin the data from the node output panel
 - The pinned data will be available without re-executing the node
-

2. Node Configuration Settings

Always Output Data

Location: Node Settings → Always Output Data

Purpose: Ensure workflow continues even when a node returns no data

Benefits:

- Prevents workflow from stopping due to empty/null responses
- Allows downstream nodes to handle empty data gracefully
- Useful for conditional logic where empty results are valid

When to enable:

- Nodes that may legitimately return empty results

- Query or filter operations that might have no matches
 - API calls that may return empty arrays/objects
-

Retry on Fail

Location: Node Settings → Retry on Fail

Best Practices:

-  **Enable for:** Operation nodes (HTTP requests, database queries, file operations)
-  **Disable for:** AI nodes (to avoid unnecessary token consumption and costs)

Configuration recommendations:

- Set appropriate retry count (typically 2-3 attempts)
 - Configure exponential backoff delays between retries
 - Use for transient failures (network issues, rate limits, temporary service unavailability)
-

3. Debugging and Monitoring

Node-Level Error Checking

Purpose: Debug and inspect specific node behavior

Implementation methods:

Option 1: Code Node with Console Logs

```
javascript

// Add a Code node after the target node
const inputData = $input.all();

console.log('=== Node Debug Info ===');
console.log('Input Data:', JSON.stringify(inputData, null, 2));
console.log('Data Type:', typeof inputData);
console.log('Is Array:', Array.isArray(inputData));
console.log('Length/Size:', inputData?.length || Object.keys(inputData).length);
console.log('=====');

return inputData;
```

Option 2: Raw Logs in VPS

- Access server logs directly for production environments
- Review timestamped execution logs
- Monitor system-level errors and warnings

When to use:

- Troubleshooting specific node failures
 - Validating data transformations
 - Performance monitoring and optimization
-

4. Error Output Configuration

On Error Setting

Location: Node Settings → On Error

Environment-specific configuration:

Development Environment

- **Setting:** "Stop Workflow"
- **Purpose:** Immediately identify and fix issues
- **Benefits:** Prevents cascading errors, easier debugging

Production Environment

- **Setting:** "Continue Using Error Output"
- **Purpose:** Maintain workflow resilience
- **Benefits:**
 - Workflow continues despite individual node failures
 - Error data is passed to error handling logic
 - Allows graceful degradation

Implementation pattern:

Node → [On Error: Continue] → Error Handler Node → Notification

5. Error Trigger Implementation

Global Error Monitoring

Purpose: Centralized error detection and notification across all workflows

Supported notification channels:

- Email notifications
- Slack messages
- Telegram alerts
- Custom webhook integrations

Configuration steps:

1. Set up Error Trigger node at workflow level
2. Configure notification preferences (email, Slack, Telegram)
3. Customize error message templates with relevant details
4. Test trigger with intentional errors

Error notification should include:

- Workflow name and ID
 - Failed node name
 - Timestamp of failure
 - Error message details
 - Execution ID for reference
-

6. AI-Assisted Debugging Process

When Errors Occur

Follow this systematic approach to leverage AI assistance:

Step 1: Capture Screenshots

Capture the following elements:

1. **Full node details** - The failed node with all settings visible

2. **Previous node** - The node immediately before the failure
3. **Complete workflow** - The entire workflow diagram showing node connections

Step 2: Gather Error Information

Collect:

- Error message text
- Input data to the failed node
- Expected vs actual output
- Node configuration settings

Step 3: Share with AI Assistant

Submit to AI platforms (ChatGPT, Claude, or others) with:

- All captured screenshots
- Error details and context
- Workflow objective description
- Specific questions about the failure

Step 4: Implement Recommendations

- Review AI-suggested solutions
 - Test fixes in development environment
 - Document the resolution for future reference
-

Quick Reference Checklist

Development Setup

- ☐ Enable "Stop Workflow" on error for all nodes
- ☐ Add debug Code nodes for critical operations
- ☐ Pin data for frequently referenced outputs
- ☐ Configure Error Trigger with test notifications

Production Setup

- ☐ Set "Continue Using Error Output" for all nodes
- ☐ Enable "Always Output Data" where appropriate

- ☐ Configure "Retry on Fail" for operation nodes (excluding AI nodes)
- ☐ Verify Error Trigger notifications are working
- ☐ Remove or disable debug Code nodes

Debugging Process

- ☐ Screenshot failed node, previous node, and workflow
 - ☐ Check raw logs if available
 - ☐ Review error output data
 - ☐ Consult AI assistant with comprehensive context
 - ☐ Document solution in workflow notes
-

Additional Best Practices

1. **Version Control:** Maintain backup versions of working workflows before making changes
 2. **Error Messages:** Use descriptive custom error messages in Code nodes
 3. **Timeout Settings:** Configure appropriate timeout values for external API calls
 4. **Data Validation:** Implement input validation before critical operations
 5. **Monitoring:** Regularly review error logs and notification history
 6. **Documentation:** Keep workflow documentation updated with error handling patterns
-

Last Updated: [Current Date]

Document Version: 1.0